

Working of Algorithm 1 and Algorithm 2

Step-by-Step Working of the Proposed Framework

- A task request arrives sequentially in the MEC system.
- Algorithm 1 first checks whether the requested task exists in the UE, Edge, or Cloud cache.
- If the task exists in the UE cache, it is executed immediately with minimum delay and energy consumption.
- If the task is not cached, the framework estimates execution latency for all processing tiers.
- The framework also estimates the corresponding energy consumption for all tiers.
- A combined execution cost is computed using latency and energy:

$$\text{Cost}_x = \alpha \hat{t}_x + \beta \hat{E}_x$$

- Only tiers satisfying the deadline constraint are considered feasible for execution.
- Among all feasible tiers, the tier with minimum execution cost is selected.
- The task is then executed at the selected processing tier (UE, Edge, or Cloud).
- After execution, the cache of the selected tier is updated.
- Algorithm 2 (Window Frequency caching) operates during this cache-update step.
- WF maintains a sliding window containing recent task requests.
- The sliding window stores only the latest requests within the predefined window size.
- If the window exceeds its size limit, the oldest request is removed.
- WF computes task frequencies only within the current sliding window.
- The cache is periodically recomputed after every R_{wf} requests.
- Tasks are ranked according to their frequencies inside the sliding window.
- The top- C most frequent tasks are retained in the cache.
- Therefore, WF preserves recently popular tasks rather than only recently used tasks.

- The updated cache directly influences future offloading decisions in Algorithm 1.
- Thus, Algorithm 1 performs:
 - online offloading,
 - deadline-aware execution,
 - latency–energy optimization.
- Algorithm 2 performs:
 - sliding-window frequency tracking,
 - short-term popularity estimation,
 - cache replacement.
- Together, both algorithms form an integrated deterministic caching–offloading MEC framework.

Python Code for Integrated Online Offloading and WF Caching

Values are taken to explain the code only:

```
from collections import deque, Counter

TIERS = ["UE", "Edge", "Cloud"]

class WindowFrequencyCache:
    def __init__(self, capacity, window_size, recompute_every):
        self.capacity = capacity
        self.window_size = window_size
        self.recompute_every = recompute_every
        self.window = deque()
        self.frequency = Counter()
        self.cache = set()
        self.counter = 0

    def contains(self, task_id):
        return task_id in self.cache

    def update(self, task_id):

        self.window.append(task_id)
        self.frequency[task_id] += 1

        if len(self.window) > self.window_size:
```

```

        old_task = self.window.popleft()

        self.frequency[old_task] -= 1

        if self.frequency[old_task] == 0:
            del self.frequency[old_task]

    self.counter += 1

    if self.counter >= self.recompute_every:

        ranked_tasks = sorted(
            self.frequency.items(),
            key=lambda x: (-x[1], x[0])
        )

        self.cache = set(
            task for task, freq in ranked_tasks[:self.capacity]
        )

        self.counter = 0

def online_offloading_and_cache_update(
    task_id,
    task_size,
    caches,
    bandwidth,
    deadline,
    alpha,
    beta,
    latency,
    compute_time,
    cache_time,
    compute_energy,
    cache_energy,
    transmission_energy
):

    cache_hit = {}

    for tier in TIERS:
        cache_hit[tier] = caches[tier].contains(task_id)

    if cache_hit["UE"]:
        return "UE", cache_time["UE"], cache_energy["UE"]

```

```

predicted_time = {}
predicted_energy = {}
cost = {}

for tier in TIERS:

    if cache_hit[tier]:

        predicted_time[tier] = cache_time[tier]
        predicted_energy[tier] = cache_energy[tier]

    else:

        predicted_time[tier] = (
            compute_time[tier]
            + latency[tier]
            + task_size / bandwidth
        )

        predicted_energy[tier] = (
            compute_energy[tier]
            + transmission_energy[tier]
        )

        cost[tier] = (
            alpha * predicted_time[tier]
            + beta * predicted_energy[tier]
        )

feasible_tiers = [
    tier for tier in TIERS
    if predicted_time[tier] <= deadline
]

if len(feasible_tiers) == 0:

    selected_tier = min(
        TIERS,
        key=lambda tier: predicted_time[tier]
    )

else:

    selected_tier = min(
        feasible_tiers,
        key=lambda tier: cost[tier]
    )

```

```

        execution_time = predicted_time[selected_tier]
        execution_energy = predicted_energy[selected_tier]

        caches[selected_tier].update(task_id)

    return selected_tier, execution_time, execution_energy

latency = {
    "UE": 0,
    "Edge": 20,
    "Cloud": 70
}

compute_time = {
    "UE": 5,
    "Edge": 20,
    "Cloud": 80
}

cache_time = {
    "UE": 1,
    "Edge": 2,
    "Cloud": 3
}

compute_energy = {
    "UE": 0.5,
    "Edge": 5.0,
    "Cloud": 50.0
}

cache_energy = {
    "UE": 0.1,
    "Edge": 0.2,
    "Cloud": 0.3
}

transmission_energy = {
    "UE": 0.0,
    "Edge": 1.0,
    "Cloud": 5.0
}

caches = {
    "UE": WindowFrequencyCache(
        capacity=3,
        window_size=5,

```

```

        recompute_every=3
    ),

    "Edge": WindowFrequencyCache(
        capacity=3,
        window_size=5,
        recompute_every=3
    ),

    "Cloud": WindowFrequencyCache(
        capacity=3,
        window_size=5,
        recompute_every=3
    )
}

tasks = [1, 2, 1, 3, 1, 2, 4, 5, 1, 2]

for step, task_id in enumerate(tasks, start=1):

    tier, delay, energy = online_offloading_and_cache_update(
        task_id=task_id,
        task_size=10,
        caches=caches,
        bandwidth=5,
        deadline=40,
        alpha=0.7,
        beta=0.3,
        latency=latency,
        compute_time=compute_time,
        cache_time=cache_time,
        compute_energy=compute_energy,
        cache_energy=cache_energy,
        transmission_energy=transmission_energy
    )

    print(f"Step {step}")
    print(f"Task ID      : {task_id}")
    print(f"Selected Tier   : {tier}")
    print(f"Execution Delay  : {delay}")
    print(f"Execution Energy : {energy}")
    print(f"UE Cache         : {sorted(caches['UE'].cache)}")
    print(f"Edge Cache       : {sorted(caches['Edge'].cache)}")
    print(f"Cloud Cache      : {sorted(caches['Cloud'].cache)}")
    print("-" * 50)

```